

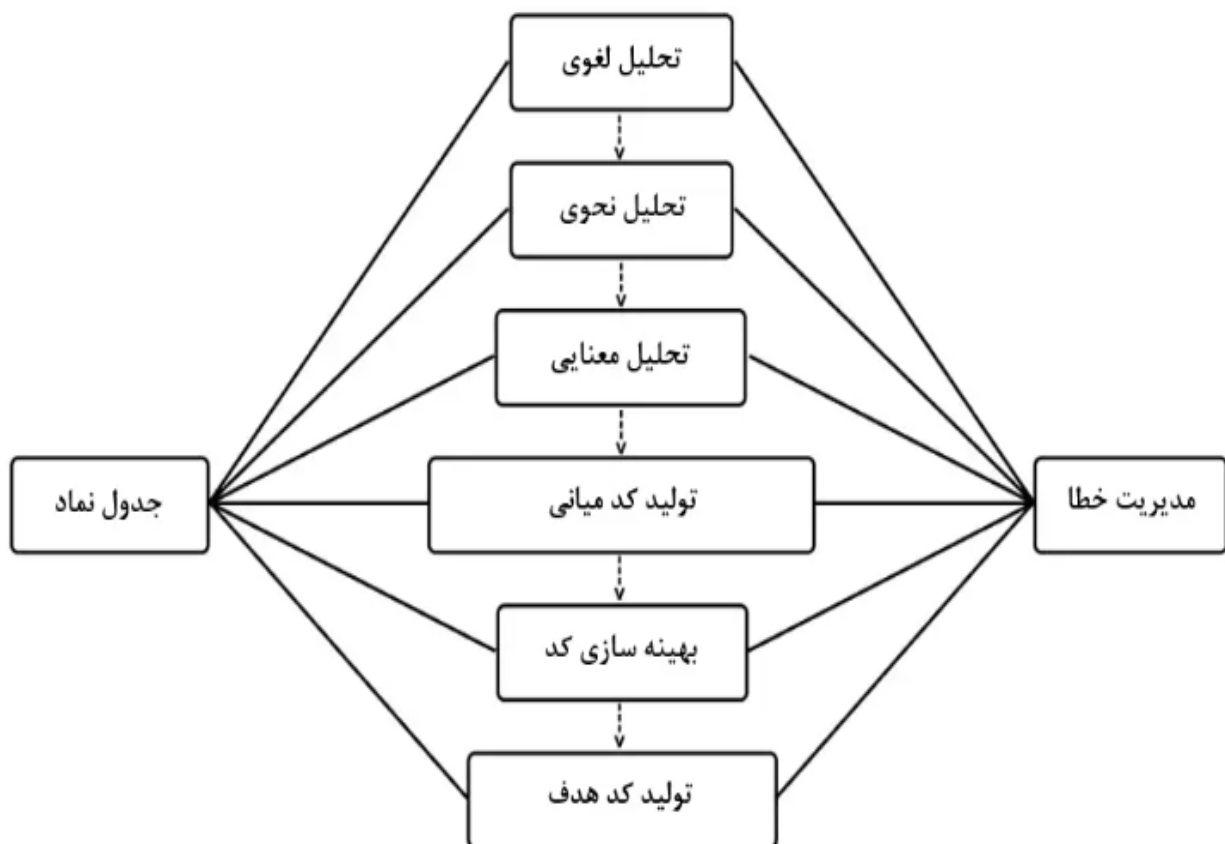
به نام خدا

پروژه درس : اصول طراحی کامپایلر

موضوع: آرایه سه بعدی

نام: مهدی خوش سیرت

کلاس: ۸ الی ۱۱





```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     // This array can store upto 12 elements (2x3x2)
6     int test[2][3][2];
7     cout << "Enter 12 values: \n";
8
9     // Inserting the values into the test array
10    // using 3 nested for loops.
11    for(int i = 0; i < 2; ++i)
12    {
13        for (int j = 0; j < 3; ++j)
14        {
15            for(int k = 0; k < 2; ++k )
16            {
17                cin >> test[i][j][k];
18            }
19        }
20    }
21    cout<<"\nDisplaying Value stored:"<<endl;
22    // Displaying the values with proper index.
23    for(int i = 0; i < 2; ++i)
24    {
25        for (int j = 0; j < 3; ++j)
26        {
27            for(int k = 0; k < 2; ++k)
28            {
29                cout << "test[" << i << "][" << j << "][" << k << "] = " << test[i][j][k]
30            }
31        }
32    }
33    return 0;
34 }
```

خروجی

Enter 12 values:

1
2
3
4
5
6
7
8
9
10
11
12

Displaying Value stored:

test[0][0][0] = 1
test[0][0][1] = 2
test[0][1][0] = 3
test[0][1][1] = 4
test[0][2][0] = 5
test[0][2][1] = 6
test[1][0][0] = 7
test[1][0][1] = 8
test[1][1][0] = 9
test[1][1][1] = 10
test[1][2][0] = 11
test[1][2][1] = 12

◆ شرح کلی عملکرد برنامه

کد بالا یک برنامه ++C است که:

1. یک آرایه سه بعدی با اندازه `test[2][3][2]` تعریف می کند (در مجموع 12 خانه).
2. از کاربر ۱۲ عدد ورودی می گیرد.
3. سپس همه مقادیر ذخیره شده در آرایه را همراه با اندیس هایشان نمایش می دهد.

◆ ارتباط با مفاهیم طراحی کامپایلر

در درس طراحی کامپایلر، ما با مراحل مختلف تحلیل سورس کد سروکار داریم:

1. تحلیل واژگانی (Lexical Analysis)
 2. تحلیل نحوی (Syntax Analysis)
 3. تحلیل معنایی (Semantic Analysis)
- و غیره.

عبارات باقاعده (Regular Expressions) ابزار اصلی مرحله تحلیل واژگانی هستند.

◆ نتیجه گیری

بنابراین، از دید طراحی کامپایلر:

- این برنامه مثالی از نحوه استفاده از ساختارهای کنترلی (حلقه های تو در تو) و آرایه ها است.
- اما اگر بخواهیم آن را از دید عبارات باقاعده تحلیل کنیم، کامپایلر برای شناسایی اجزای کد (کلیدواژه ها، شناسه ها، عملگرها، عددها و رشته ها) از عبارات باقاعده در مرحله تحلیل واژگانی (Lexical Analysis) استفاده می کند.



Line	Lexeme	Type	Value	Token
1	int	Kw	int	<int, >
2	Test	ID	Test	<ID, 2>
3	[	S	[	<[, >
4	2	Num	2	<2, >
5	]	S	]	<], >
6	[	S	[	<[, >
7	3	Num	3	<3, >
8	]	S	]	<], >
9	[	S	[	<[, >
10	2	Num	2	<2, >
11	]	S	]	<], >
12	;	S	;	<;, >
13	cout	Kw	cout	<cout, >
14	<<	S	<<	<<<, >
15	"	STR	"	<" , >
16	Enter 12 values: \n	STR	Enter 12 values: \n	<Enter, >
17	"	S	"	<" , >
18	;	S	;	<;, >
19	for	Kw	for	<for, >
20	(	S	(	<[, >
21	int	Kw	int	<int, >
22	j	ID	j	<ID, 22>
23	=	op	=	<=, >
24	0	Num	0	<0, >
25	;	S	;	<;, >

~~~~~

|    |     |     |     |          |
|----|-----|-----|-----|----------|
| r9 | i   | ID  | i   | <ID, r9> |
| rV | <   | op  | <   | <<, >    |
| rA | r   | num | r   | <r, >    |
| r9 | ;   | s   | ;   | <:, >    |
| r0 | +   | op  | +   | <+, >    |
| r1 | +   | op  | +   | <+, >    |
| r2 | i   | ID  | i   | <ID, r2> |
| r2 | )   | s   | )   | <), >    |
| rE | for | kw  | for | <for, >  |
| rA | (   | s   | (   | <(, >    |
| r9 | int | kw  | int | <int, >  |
| rV | j   | ID  | j   | <ID, rV> |
| rA | =   | op  | =   | <=, >    |
| r9 | o   | num | o   | <o, >    |
| E0 | ;   | s   | ;   | <:, >    |
| E1 | j   | ID  | j   | <ID, E1> |
| E2 | <   | op  | <   | <<, >    |
| E3 | r   | num | r   | <r, >    |
| EE | ;   | s   | ;   | <:, >    |
| EΔ | +   | op  | +   | <+, >    |
| F4 | +   | op  | +   | <+, >    |
| EV | j   | ID  | j   | <ID, EV> |
| FA | )   | s   | )   | <), >    |
| E9 | for | kw  | for | <for, >  |
| Q0 | (   | s   | (   | <(, >    |



Date \_\_\_\_\_

|    |      |     |      |          |
|----|------|-----|------|----------|
| ۱  | int  | kw  | int  | <int, >  |
| ۲  | K    | ID  | K    | <ID, ۲>  |
| ۳  | =    | op  | =    | <=, >    |
| ۴  | ۰    | num | ۰    | <۰, >    |
| ۵  | ;    | s   | ;    | <;, >    |
| ۶  | K    | ID  | K    | <ID, ۶>  |
| ۷  | <    | op  | <    | <<, >    |
| ۸  | ;    | s   | ;    | <;, >    |
| ۹  | +    | op  | +    | <+, >    |
| ۱۰ | +    | op  | +    | <+, >    |
| ۱۱ | K    | ID  | K    | <ID, ۱۱> |
| ۱۲ | )    | s   | )    | <), >    |
| ۱۳ | cin  | kw  | cin  | <cin, >  |
| ۱۴ | >>   | s   | >>   | <>>, >   |
| ۱۵ | Test | ID  | Test | <ID, ۱۵> |
| ۱۶ | [    | s   | [    | <[, >    |
| ۱۷ | i    | ID  | i    | <ID, ۱۷> |
| ۱۸ | ]    | s   | ]    | <], >    |
| ۱۹ | [    | s   | [    | <[, >    |
| ۲۰ | j    | ID  | j    | <ID, ۲۰> |
| ۲۱ | ]    | s   | ]    | <], >    |
| ۲۲ | [    | s   | [    | <[, >    |
| ۲۳ | K    | ID  | K    | <ID, ۲۳> |
| ۲۴ | ]    | s   | ]    | <], >    |
| ۲۵ | ;    | s   | ;    | <;, >    |

در ادامه همین کار دوباره تکرار شود

| نوع توکن (Token Type)                 | مثال در کد                                                    | عبارت باقاعده (Regular Expression)                | توضیح                                          |
|---------------------------------------|---------------------------------------------------------------|---------------------------------------------------|------------------------------------------------|
| کلمه کلیدی (Keyword)                  | <code>int, for, return, using, namespace, cout, cin</code>    | <code>b(int\`</code>                              | <code>for</code>                               |
| شناسه (Identifier)                    | <code>main, test, i, j, k, std</code>                         | <code>*[_a-zA-Z0-9][_a-zA-Z]</code>               | نام‌هایی که برنامه‌نویس تعریف می‌کند           |
| عدد صحیح (Integer Literal)            | <code>12, 3, 2, 0</code>                                      | <code>+ [0-9]</code>                              | شامل ارقام عددی بدون اعشار                     |
| علامت عملگر انتساب و مقایسه           | <code>++, &gt;, =</code>                                      | <code>=)\`</code>                                 | <code>&gt;</code>                              |
| علامت‌های پرانتز و کروشه              | <code>[, ], {, }, (, )</code>                                 | <code>[ [ \ ] \ { \ } \ ( \ ) \ ]</code>          | برای گروه‌بندی و آرایه‌ها                      |
| نقطه‌ویرگول (semicolon)               | <code>;</code>                                                | <code>;</code>                                    | پایان‌دهنده‌ی دستورات                          |
| علامت دو نقطه و عملگرهای جریان        | <code>&lt;&lt;, &gt;&gt;</code>                               | <code>&gt;&gt;)\`</code>                          | <code>&lt;&lt;</code>                          |
| رشته (String Literal)                 | <code>Enter 12 values: \n", "\nDisplaying Value stored</code> | <code>[ \"^ ) ] \"</code>                         | <code>\"*(\`</code>                            |
| فضای خالی (Whitespace)                | فاصله، تب، خط جدید                                            | <code>+ [t \n \r \ ]</code>                       | بین توکن‌ها قرار می‌گیرد (نادیده گرفته می‌شود) |
| پیش‌پردازنده (Preprocessor Directive) | <code>&lt;include &lt;iostream#</code>                        | <code>&lt;+ [ &lt; ^ ] &gt; * include \s #</code> | دستورهای کامپایل قبل از اجرا                   |

♦ در پایان همه‌ی عبارات باقاعده پشت سر هم:

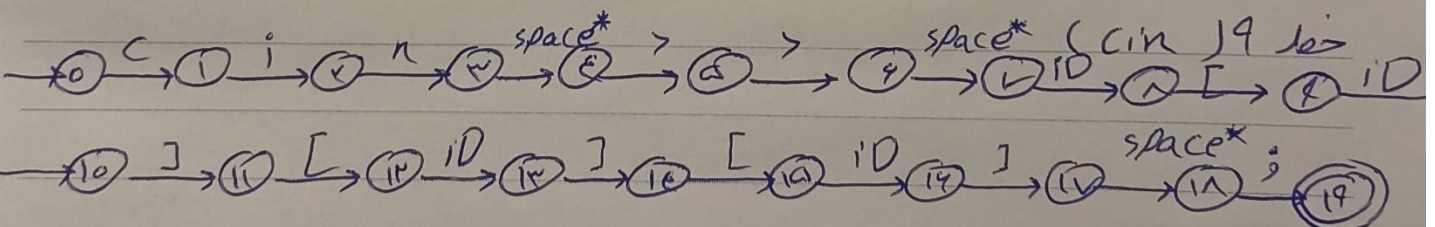
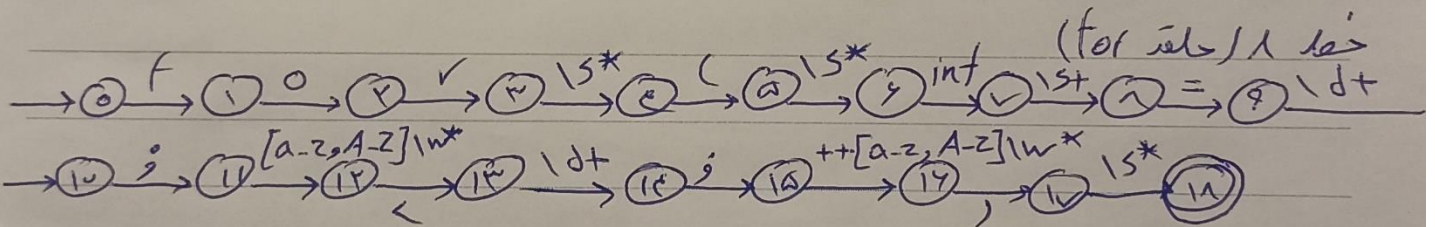
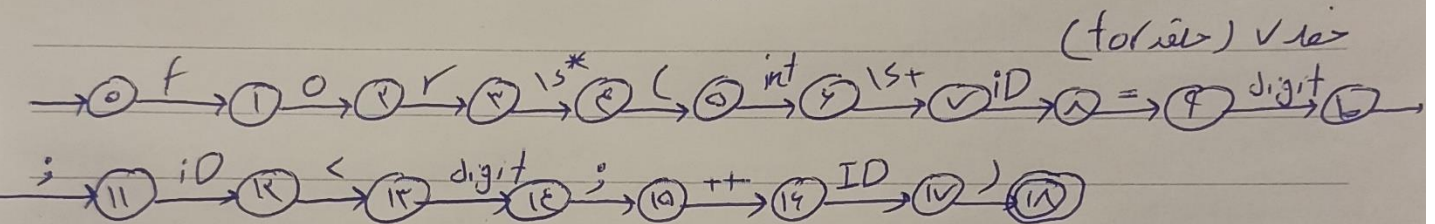
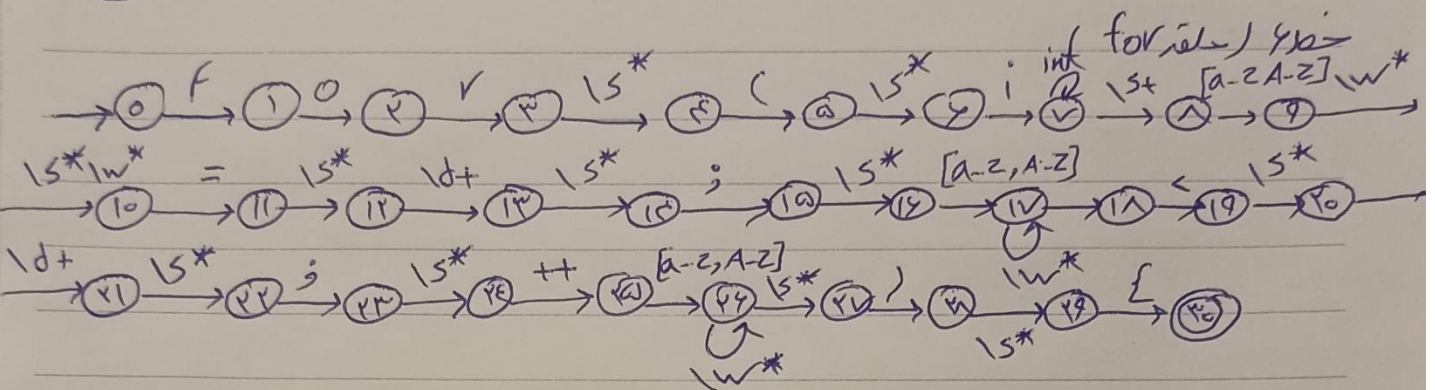
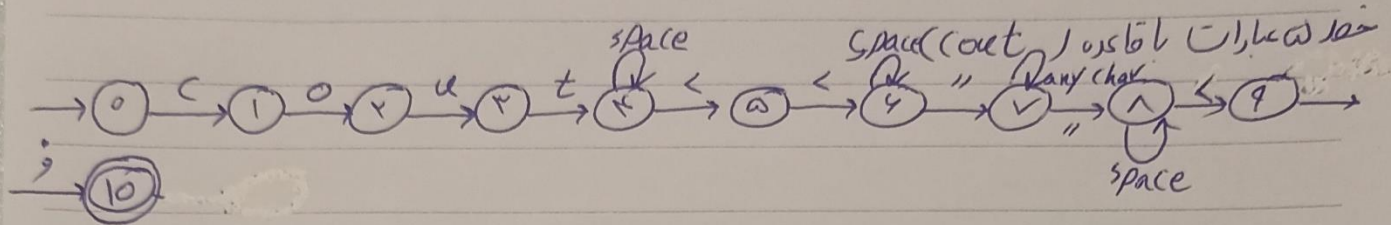
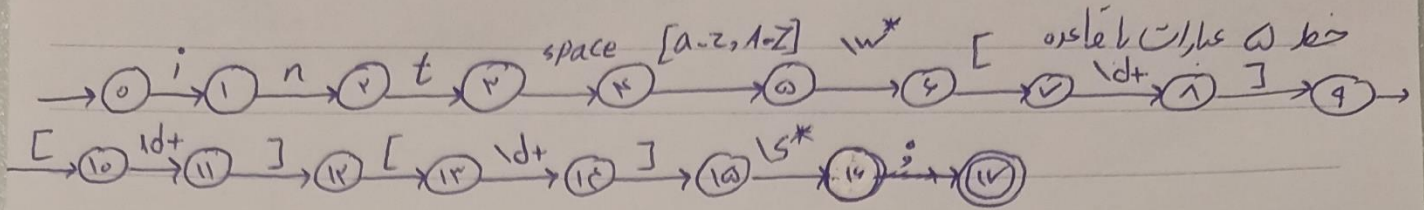
Copy code

markdown

```
^#include\s*<\w+(\. \w+)*>$
^using\s+namespace\s+[_a-zA-Z_]\w*\s*;$
^(int/void)\s+main\s*\(\s*\)\s*$
^\{ $
^int\s+[_a-zA-Z_]\w*(\[\d+\])\{3\}\s*;$
^cout\s*<<\s*".*" \s*<<?\s*;$
^for\s*\(\s*int\s+[_a-zA-Z_]\w*\s*=\s*\d+\s*;\s*[_a-zA-Z_]\w*\s*<\s*\d+\s*;\s*|+[_a-zA-Z_]\w*\s*\)\s*
^\{ $
^for\s*\(\s*int\s+[_a-zA-Z_]\w*\s*=\s*\d+\s*;\s*[_a-zA-Z_]\w*\s*<\s*\d+\s*;\s*|+[_a-zA-Z_]\w*\s*\)\s*
^\{ $
^for\s*\(\s*int\s+[_a-zA-Z_]\w*\s*=\s*\d+\s*;\s*[_a-zA-Z_]\w*\s*<\s*\d+\s*;\s*|+[_a-zA-Z_]\w*\s*\)\s*
^\{ $
^cin\s*>>\s*[_a-zA-Z_]\w*(\[[a-zA-Z_]\w*\)\{3\}\s*;$
^\} $
```

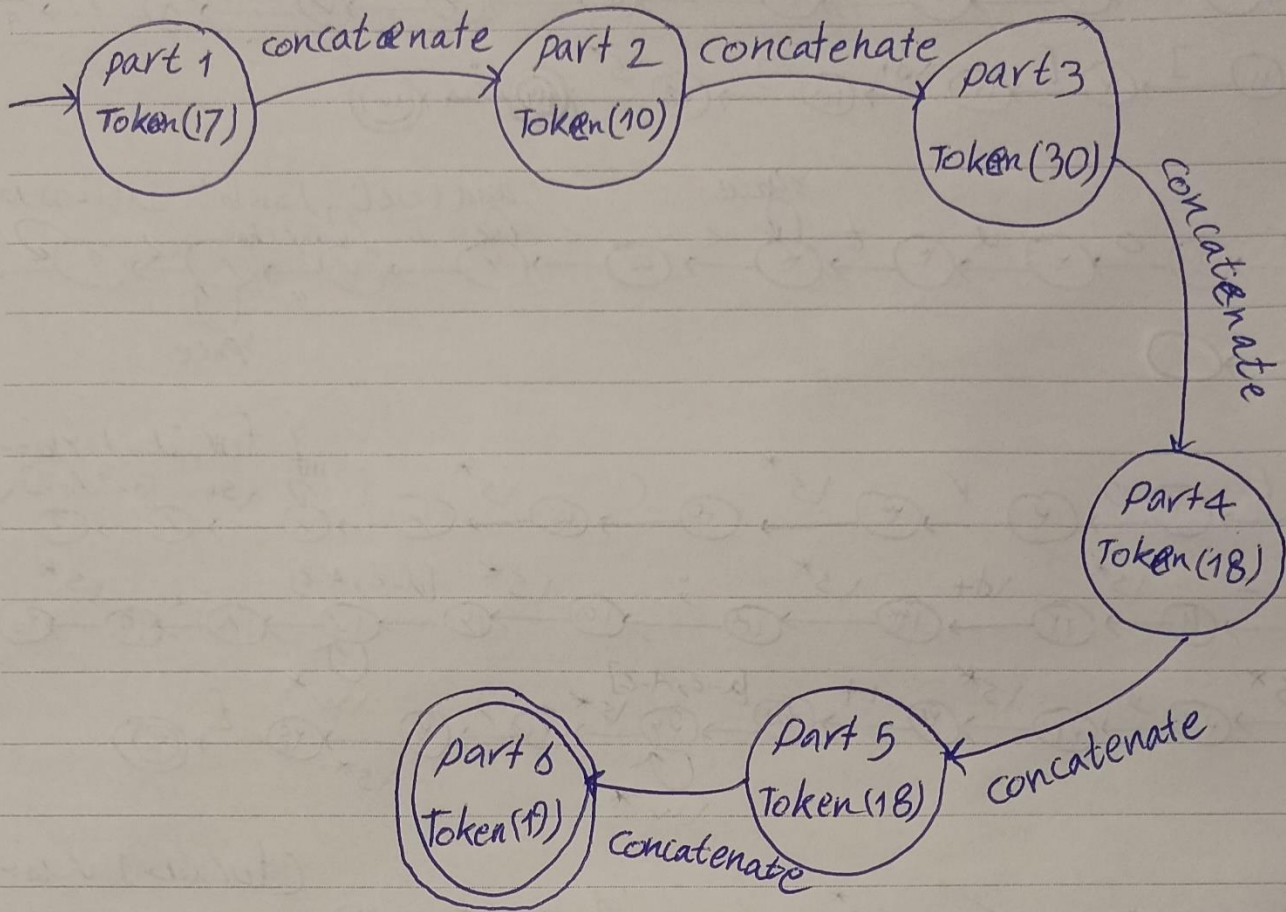
نوع 1 - فاصله ها، ستاره 15\* - اولویت 15\* - 1d+ - 1d+

نوع 2 -





نمودار DFA



## 💡 تفسیر Regex ها به ساختار نحوی

به صورت خلاصه، Regex ها این اجزاء را توصیف می کنند:

1. تابع `$* main ^(int|void)\s+main\s*(\s*\s)\s`

2. آکولاد باز و بسته `{ }` و `{ }`

3. تعریف آرایه سه بعدی: `$;*int\s+[a-zA-Z_]\w*(\[\d+\]){3}\s^`

4. دستور خروجی `$;* cout :^cout\s*<<\s*".*"\s*<<?\s`

5. حلقه های `for` تو در تو: `for\s*(\s*int\s+[a-zA-Z_]\w*\s*=\s*\d+\s*; \s*[a-zA-Z_]\w*\s*<\s*\d+\s*; \s*\+|^`

`$*[a-zA-Z_]\w*\s*\s`

6. ورودی با `$;* cin :^cin\s*>>\s*[a-zA-Z_]\w*(\[[a-zA-Z_]\w*\]){3}\s`

7. آکولاد بسته در پایان

## 📘 گرامر (CFG) پیشنهادی به صورت کامل

در قالب شبه-BNF یا EBNF:

`<Program> ::= <MainFunction>`

`<MainFunction> ::= "int" "main" "(" ")" "{" <Body> "}"`

`<Body> ::= <ArrayDecl> <CoutStmt> <ThreeNestedFors>`

`<ArrayDecl> ::= "int" <IDENT> <ArraySize3> ";"`

`<ArraySize3> ::= "[" <NUMBER> "]" "[" <NUMBER> "]" "[" <NUMBER> "]"`

`<CoutStmt> ::= "cout" "<<" <STRING> [ "<<" <Expression> ] ";"`

`<ThreeNestedFors> ::= <ForHeader> "{" <ForHeader> "{" <ForHeader> "{" <InnerStmt> "}" "}" "}"`

`<ForHeader> ::= "for" "(" "int" <IDENT> "=" <NUMBER> ";" <IDENT> "<" <NUMBER> ";" "++" <IDENT> ")"`

`<InnerStmt> ::= <CinStmt>`

`<CinStmt> ::= "cin" ">>" <IDENT> "[" <IDENT> "]" "[" <IDENT> "]" "[" <IDENT> "]" ";"`

## تعاریف لغوی (توکن‌ها — با regex)

- `* IDENT> := [A-Za-z_][A-Za-z0-9_]>`
- `+ NUMBER> := [0-9]>`
- `"*(.\\|\\\"^):?)" =: <STRING>`

🎯 مثال یک نمونه کد منطبق با این گرامر:

cpp



```
int main() {
    int arr[10][10][10];
    for (int i = 0; i < 10; ++i) {
        for (int j = 0; j < 10; ++j) {
            for (int k = 0; k < 10; ++k) {
                cin >> arr[i][j][k];
            }
        }
    }
    cout << "Done!";
}
```

Program

```

├─ FunctionHeader
|   ├─ "int"
|   ├─ "main"
|   ├─ "("
|   └─ ")"
├─ "{"
├─ Declarations
|   └─ VarDecl
|       ├─ "int"
|       ├─ Identifier → "arr"
|       ├─ "[" Number "]" → [10]
|       ├─ "[" Number "]" → [10]
|       ├─ "[" Number "]" → [10]
|       └─ ";"
├─ Loops
|   └─ ForLoop (i)
|       ├─ "for" "(" "int" Identifier "=" Number ";" Identifier "<" Number ";" "++" Identifier ")" "{"
|       |   └─ Identifier: "i"
|       |       └─ Number: 0 , Number: 10
|       |   └─ Inner
|       |       └─ ForLoop (j)
|       |           ├─ "for" "(" "int" Identifier "=" Number ";" Identifier "<" Number ";" "++" Identifier ")" "{"
|       |           |   └─ Identifier: "j"
|       |           |       └─ Number: 0 , Number: 10
|       |           |   └─ Inner
|       |           |       └─ ForLoop (k)
|       |           |           ├─ "for" "(" "int" Identifier "=" Number ";" Identifier "<" Number ";" "++" Identifier ")" "{"
|       |           |           |   └─ Identifier: "k"
|       |           |           |       └─ Number: 0 , Number: 10
|       |           |           |   └─ InnerStatements
|       |           |           |       └─ CinStatement
|       |           |           |           ├─ "cin"
|       |           |           |           ├─ ">>"
|       |           |           |           ├─ Identifier → "arr"
|       |           |           |           ├─ "[" Identifier "]" "[" Identifier "]" "[" Identifier "]"
|       |           |           |           |   └─ i , j , k
|       |           |           |           └─ ";"
|       |           |           └─ "}"
|       |           └─ "}"
|       └─ "}"
├─ IOStatements
|   └─ CoutStatement
|       ├─ "cout"
|       ├─ "<<" StringLiteral → "Done!"
|       └─ ";"
└─ "}"

```



## توضیح گام به گام درخت

1. ریشه: Program

○ آغازگر تجزیه کل برنامه است.

2. FunctionHeader:

○ شامل نوع `int` و نام تابع `main()`.

3. Declarations:

○ یک آرایه سه بعدی تعریف می شود (مطابق Rule VarDec1).

4. Loops:

○ شامل سه حلقه تو در توست (ForLoop X3)، که هرکدام شمارنده مخصوص دارند (i, j, k).

5. InnerStatements:

○ داخل حلقه آخر، دستور ورودی `cin >> arr[i][j][k]` قرار دارد.

6. IOStatements (در انتهای برنامه):

○ دستور خروجی `cout << "Done";`

7. آکولاد پایانی:

○ نشانگر پایان محدوده تابع است.

Program

```
|
|— FunctionHeader
|   |— int
|   |— main()
|
|— {
|
|— Declarations
|   |— int arr[10][10][10];
|
|— Loops
|   |— for (int i=0; i<10; ++i)
|       |— for (int j=0; j<10; ++j)
|           |— for (int k=0; k<10; ++k)
|               |— cin >> arr[i][j][k];
|
|— cout << "Done!";
|
|— }
```

# رسم جدول تجزیه

## 1. نمادهای گرامر

نمادهای غیر پایانی (Non-Terminals):

Copy code

CSS

```
Program, MainFunction, Body, ArrayDecl, ArraySize3, CoutStmt, ThreeNestedFors,
ForHeader, InnerStmt, CinStmt
```

نمادهای پایانی (Terminals):

Copy code

cpp

```
int, main, (, ), {, }, ;, [, ], cout, <<, cin, >>, for, =, <, ++, IDENT, NUMBER, STRING
```

## مجموعه‌های FIRST و FOLLOW (خلاصه)

### FIRST

- `FIRST(Program) = { "int" }`
- `FIRST(MainFunction) = { "int" }`
- `FIRST(Body) = { "int" }`
- `FIRST(ArrayDecl) = { "int" }`
- `{ "]" } = FIRST(ArraySize3)`
- `FIRST(CoutStmt) = { "cout" }`
- `FIRST(CoutOpt) = { "<<",  $\epsilon$  }`
- `FIRST(ThreeNestedFors) = { "for" }`
- `FIRST(ForHeader) = { "for" }`
- `FIRST(InnerStmt) = { "cin" }`
- `FIRST(CinStmt) = { "cin" }`

### FOLLOW

- `{ "$" } = FOLLOW(Program)` (علامت پایان ورودی)
- `{ "$" } = FOLLOW(MainFunction)`
- `{ "{" } = FOLLOW(Body)`
- `FOLLOW(ArrayDecl) = { "cout" }` (بعد از Body در ArrayDecl ترمینال cout می‌آید)
- `{ ";" } = FOLLOW(ArraySize3)`
- `FOLLOW(CoutStmt) = { "for" }`
- `{ ";" } = FOLLOW(CoutOpt)` (CoutOpt قبل از ; قرار دارد)
- `{ "{" } = FOLLOW(ThreeNestedFors)`
- `{ "]" } = FOLLOW(ForHeader)`
- `{ "{" } = FOLLOW(InnerStmt)`
- `{ "{" } = FOLLOW(CinStmt)`

## شماره‌گذاری تولیدها (برای ارجاع در جدول)

1. Program → MainFunction
2. "MainFunction → "int" "main" "(" " ")" "{" Body "}"
3. Body → ArrayDecl CoutStmt ThreeNestedFors
4. ";" ArrayDecl → "int" IDENT ArraySize3
5. "ArraySize3 → "[" NUMBER "]" "[" NUMBER "]" "[" NUMBER "]"
6. ";" CoutStmt → "cout" "<<" STRING CoutOpt
7. CoutOpt → "<<" IDENT
8. CoutOpt → ε
9. "ThreeNestedFors → ForHeader "{" ForHeader "{" ForHeader "{" InnerStmt "}" "}" "}"
10. "ForHeader → "for" "(" "int" IDENT "=" NUMBER ";" IDENT "<" NUMBER ";" "++" IDENT ")"
11. InnerStmt → CinStmt
12. ";" CinStmt → "cin" ">>" IDENT "[" IDENT "]" "[" IDENT "]" "[" IDENT "]"

## جدول تجزیه

| \$ | ;     | >>             | ]                                          | cin                                                    | for                                                                | cout                                 | int                                          | Nonterminal \ lookahead |
|----|-------|----------------|--------------------------------------------|--------------------------------------------------------|--------------------------------------------------------------------|--------------------------------------|----------------------------------------------|-------------------------|
| —  | —     | —              | —                                          | —                                                      | —                                                                  | —                                    | Program → 1<br>(MainFunction                 | Program                 |
| —  | —     | —              | —                                          | —                                                      | —                                                                  | —                                    | int main(...) { } 2<br>( Body )              | MainFunction            |
| —  | —     | —              | —                                          | —                                                      | —                                                                  | —                                    | ArrayDecl) 3<br>CoutStmt<br>(ThreeNestedFors | Body                    |
| —  | —     | —              | —                                          | —                                                      | —                                                                  | —                                    | int IDENT ) 4<br>( ; ArraySize3              | ArrayDecl               |
| —  | —     | —              | [ NUMBER ] 5<br>[ NUMBER ]<br>( [ NUMBER ] | —                                                      | —                                                                  | —                                    | —                                            | ArraySize3              |
| —  | —     | —              | —                                          | —                                                      | —                                                                  | cout << ) 6<br>STRING CoutOpt<br>( ; | —                                            | CoutStmt                |
| —  | (ε) 8 | ( IDENT >> ) 7 | —                                          | —                                                      | —                                                                  | —                                    | —                                            | CoutOpt                 |
| —  | —     | —              | —                                          | —                                                      | ForHeader { ForHeader) 9<br>{ ForHeader { InnerStmt<br>( } } }     | —                                    | —                                            | ThreeNestedFors         |
| —  | —     | —              | —                                          | —                                                      | for ( int IDENT = ) 10<br>NUMBER ; IDENT < NUMBER ;<br>( ++IDENT ) | —                                    | —                                            | ForHeader               |
| —  | —     | —              | —                                          | (InnerStmt → CinStmt) 11                               | —                                                                  | —                                    | —                                            | InnerStmt               |
| —  | —     | —              | —                                          | cin >> IDENT [ IDENT ] ) 12<br>( ; [ IDENT ] [ IDENT ] | —                                                                  | —                                    | —                                            | CinStmt                 |